IoT LAB (EC2601-1)

A Project Report on

Air & Noise Pollution Monitoring using Raspberry Pi Pico

Submitted By:

Sampath, NNM23EC153
Samruddha, NNM23EC154
Samruddhi S S, NNM23EC155

NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY
Department of Electronics & Communication Engineering

May – 2026

NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Department of Electronics & Communication Engineering

CERTIFICATE

This is to certify that ***Sampath (NNM23EC153), Samruddha (NNM23EC154) and Samruddhi S S (NNM23EC155)***, are bonafide students of N.M.A.M. Institute of Technology, Nitte, who have submitted a project report entitled "**Air & Noise Monitoring System using Raspberry Pi Pico** " as part of the 6^{th} semester IoT Lab, in partial fulfillment of the requirements for the award of Bachelor of Engineering Degree in Electronics and Communication Engineering during the year 2025–2026.

Name of the Examiners	**Signature with date**
1._____	_____
2._____	_____

ABSTRACT

This project presents the design and implementation of a low-cost, real-time, multi-parameter environmental monitoring system using the Raspberry Pi Pico W microcontroller, aimed at simultaneously measuring air quality and ambient noise pollution in urban environments. The system integrates an MQ135 gas sensor to detect Volatile Organic Compounds (VOCs) and CO2 equivalents, and a sound sensor module to quantify noise levels in decibels (dB), with both sensors interfaced via the 12-bit ADC of the Raspberry Pi Pico W. On-board MicroPython firmware processes the acquired sensor data and transmits it wirelessly to the ThingSpeak cloud IoT platform using the MQTT (Message Queuing Telemetry Transport) protocol over the built-in Wi-Fi interface, enabling real-time dashboard visualization, historical trend analysis, and threshold-based alerting. The MQ135 sensor operates on the principle of chemiresistance, where its internal resistance varies with the concentration of target gases, and the resulting analog voltage is sampled by the ADC and mapped to a corresponding air quality index value. Similarly, the sound sensor module converts acoustic pressure variations into proportional analog voltage signals, which are digitized and mathematically processed to derive decibel readings. The entire data acquisition cycle runs at periodic intervals defined in the MicroPython firmware, ensuring consistent and reliable sampling without overloading the MQTT broker or exceeding ThingSpeak's data update rate limits. Unlike conventional single-parameter monitors, this system provides a correlated, holistic view of environmental health from a single compact and affordable node, making it well-suited for deployment in smart city infrastructure, residential areas, and educational IoT research.

Contents

Chapter 1

INTRODUCTION

1.1 Background

Rapid urbanization and industrial growth have significantly worsened air and noise pollution in urban areas. Prolonged exposure to harmful gases like VOCs and CO2 causes respiratory and cardiovascular disorders, while excessive noise leads to stress, hearing impairment, and sleep disturbances. Traditional monitoring systems are expensive, bulky, and limited in coverage, making community-level real-time monitoring difficult. The IoT paradigm enables low-cost, networked sensor nodes that can continuously collect and transmit environmental data to cloud platforms, offering a practical and scalable alternative. The growing availability of low-cost microcontrollers with built-in wireless connectivity, such as the Raspberry Pi Pico W, has further accelerated the adoption of IoT-based environmental monitoring solutions. These systems are capable of operating continuously with minimal power consumption, making them suitable for long-term deployment in both indoor and outdoor environments. Furthermore, cloud platforms like ThingSpeak provide powerful tools for data aggregation, visualization, and automated alerting, enabling authorities and researchers to make data-driven decisions regarding environmental health and urban planning.

1.2 Problem Statement

Rapid urbanization and industrial growth have significantly worsened air and noise pollution in urban areas. Prolonged exposure to harmful gases like VOCs and CO2 causes respiratory and cardiovascular disorders, while excessive noise leads to stress, hearing impairment, and sleep disturbances. Traditional monitoring systems are expensive, bulky, and limited in coverage, making community-level real-time monitoring difficult. The IoT paradigm enables low-cost, networked sensor nodes that can continuously collect and transmit environmental data to cloud platforms, offering a practical and scalable alternative. The growing availability of low-cost microcontrollers with built-in wireless connectivity, such as the Raspberry Pi Pico W, has further accelerated the adoption of IoT-based environmental monitoring solutions. These systems are capable of operating continuously with minimal power consumption, making them suitable for long-term deployment in both indoor and outdoor environments. Furthermore, cloud platforms like ThingSpeak provide powerful tools for data aggregation, visualization, and automated alerting, enabling authorities and researchers to make data-driven decisions regarding environmental health and urban planning.

1.3 Objectives

1. To measure air quality (VOCs/CO2 equivalents) using the MQ135 sensor via the Raspberry Pi Pico.
2. To quantify ambient noise levels using a sound sensor module.
3. To implement MicroPython firmware for on-board data acquisition and processing.
4. To transmit sensor data wirelessly to ThingSpeak using the MQTT protocol over built-in Wi-Fi.
5. To visualize real-time data and trigger threshold-based alerts on the ThingSpeak dashboard.
6. To develop a low-cost, compact, and scalable monitoring node suitable for deployment in smart city networks and educational IoT research environments.
7. To correlate air quality and noise pollution data from a single node to provide a holistic view of urban environmental health.

1.4 Scope

The project covers the design, prototyping, and validation of a dual-parameter environmental monitoring node suitable for indoor and outdoor deployment in smart city networks, residential areas, and educational IoT labs. The system is designed to operate continuously and transmit data at regular intervals to the ThingSpeak cloud platform, where it can be monitored and analyzed remotely. The project demonstrates the feasibility of using the Raspberry Pi Pico W as a cost-effective and capable IoT node for environmental sensing applications. It does not include PM2.5/PM10 particulate sensing, GPS geotagging, or machine learning-based prediction, which are reserved as future enhancements. Additionally, the system currently supports single-node deployment and does not implement a multi-node mesh network architecture, which can be considered as a future extension to cover larger geographical areas.

Chapter 2

LITERATURE REVIEW

2.1 Write the existing methods

Early IoT-based environmental monitoring systems used Arduino platforms with MQ-series sensors and basic HTTP communication, which lacked wireless connectivity and cloud scalability. These systems were primarily limited to local data logging with no provision for remote access or real-time alerting, making them unsuitable for continuous environmental monitoring applications. The introduction of Wi-Fi-enabled microcontrollers like the ESP8266 and ESP32 marked a significant improvement, enabling cloud-integrated solutions where MQ135-based air quality monitoring and LM393-based noise sensing were individually demonstrated with reasonable accuracy. The shift to MQTT protocol further improved these systems by providing lightweight, low-bandwidth, publish-subscribe communication suitable for resource-constrained IoT nodes, with platforms like ThingSpeak and Adafruit IO serving as cloud backends for visualization and threshold-based alerting. Several research works proposed air quality monitoring systems using MQ-series sensors transmitting data to ThingSpeak or AWS IoT, demonstrating the viability of low-cost gas sensors for detecting VOCs and CO2 in urban environments. However, most of these works did not address sensor calibration with respect to temperature and humidity variations, affecting the reliability of readings in real-world deployments. Similarly, noise pollution monitoring systems using sound sensor modules were developed as standalone solutions without integration with air quality sensing, limiting their ability to provide a holistic assessment of environmental health. The Raspberry Pi Pico W, introduced in June 2022, features a dual-core ARM Cortex-M0+ processor, a 12-bit ADC, and an onboard Wi-Fi module, making it a capable and cost-effective platform for IoT applications. However, most existing works using the Pico W are limited to single-sensor demonstrations without cloud integration. This project directly addresses that gap by integrating both air quality and noise sensing on the Raspberry Pi Pico W with full MQTT-based ThingSpeak cloud connectivity, providing a more comprehensive and practical environmental monitoring solution compared to existing works.

Chapter 3

METHODOLOGY

3.1 Block diagram / Flowchart

The system is built around the Raspberry Pi Pico W as the central MCU, which interfaces with the MQ135 Gas Sensor (via ADC pin) to measure air quality and VOC levels, and a Sound Sensor Module to capture ambient noise levels in decibels. The firmware is developed using MicroPython in Thonny IDE, where the code handles sensor initialization, ADC data reading, WiFi connectivity, and MQTT communication. The collected sensor data is published at regular intervals to the ThingSpeak cloud platform via the MQTT protocol over WiFi, where it is stored, visualized as real-time graphs, and monitored for threshold-based alerts to indicate hazardous pollution levels.

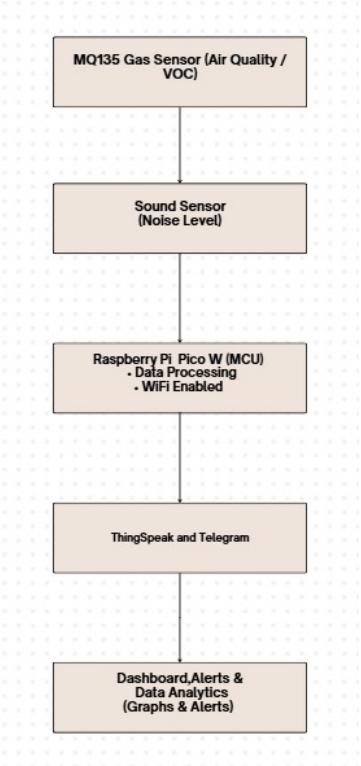


Figure 3.1: Block Diagram

3.2 Working Principle

The Air and Noise Pollution Monitoring System operates by continuously sensing environmental parameters through two key sensors connected to the Raspberry Pi Pico W. The overall working of the system can be understood in three stages: sensor data acquisition, data processing, and cloud transmission.

Stage 1: Sensor Data Acquisition The MQ135 Gas Sensor detects harmful gases such as VOCs and CO2 equivalents by varying its electrical resistance in response to gas concentration. The sensor operates on the principle of chemiresistance, where the sensitive material inside the sensor undergoes a change in electrical resistance when it comes into contact with target gas molecules. This resistance change produces a corresponding change in the output voltage of the sensor, which is fed to the ADC pin of the Raspberry Pi Pico W. The 12-bit ADC of the Pico W samples this analog voltage and converts it into a digital value ranging from 0 to 4095, which is then mapped to a corresponding air quality index value through a mathematical formula defined in the MicroPython firmware. Simultaneously, the Sound Sensor Module captures ambient sound waves through a condenser microphone and passes the signal through an op-amp based signal conditioning circuit that amplifies the weak microphone output to a measurable analog voltage level. This amplified analog voltage, which is proportional to the ambient noise intensity, is sampled by another ADC channel of the Raspberry Pi Pico W and mathematically mapped to a decibel (dB) value using a predefined conversion formula in the firmware.

Stage 2: Data Processing Once the raw ADC values are acquired from both sensors, the MicroPython firmware running on the Raspberry Pi Pico W processes and converts them into meaningful physical quantities. The air quality ADC value is mapped to a VOC or CO2 equivalent concentration level, and the sound sensor ADC value is converted to an ambient noise level in decibels. The firmware also performs basic validation checks on the sensor readings to filter out any erroneous or out-of-range values before transmission. The processed data is then formatted into a suitable payload structure for MQTT transmission to the ThingSpeak cloud platform.

Stage 3: Cloud Transmission and Visualization The Raspberry Pi Pico W connects to a local WiFi network using its onboard CYW43439 wireless module and establishes an MQTT client connection with the ThingSpeak MQTT broker. The processed sensor data is published to dedicated ThingSpeak channel fields at regular time intervals using the MQTT publish mechanism. ThingSpeak receives the incoming data, stores it in its cloud database, and renders it as real-time graphical plots on the dashboard for remote monitoring and analysis. Additionally, ThingSpeak's threshold-based alerting mechanism triggers notifications whenever the air quality index or noise level exceeds predefined safe limits, enabling timely response to hazardous environmental conditions.
This three-stage working principle ensures a continuous, reliable, and real-time environmental monitoring pipeline from sensor to cloud.

Chapter 4

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 Hardware Requirements

1. Raspberry Pi Pico W (with onboard WiFi) — Central Microcontroller Unit
2. MQ135 Gas Sensor — Air Quality and VOC detection
3. Sound Sensor Module (Analog Output) — Ambient Noise Level measurement
4. Micro-USB Cable — For powering and programming the Pico W
5. Connecting Wires and Breadboard — For circuit connections

4.2 Software Requirements

1. Thonny IDE — For writing and uploading MicroPython code
2. MQTT Protocol — For lightweight data transmission to cloud
3. ThingSpeak Cloud Platform — For data storage, visualization and monitoring
4. Telegram Bot — For sending alert messages.

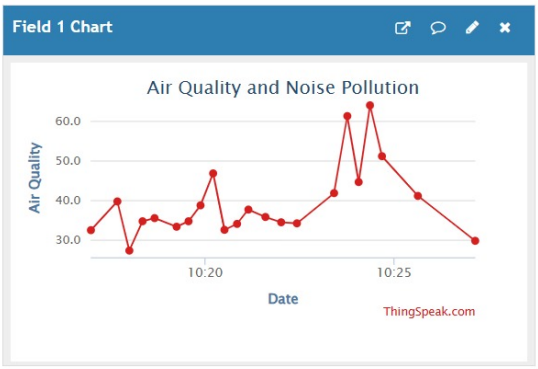
Figure 4.1: Hardware Setup

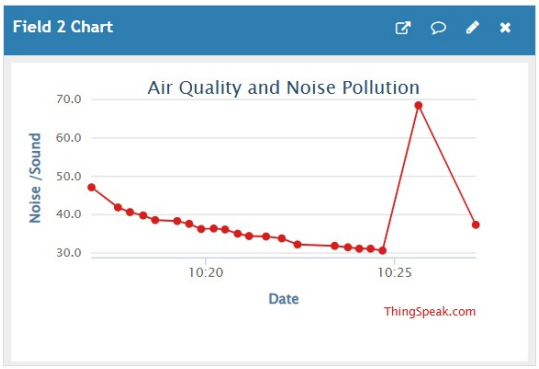
Chapter 5

RESULTS

5.1 ThingSpeak Charts

Two real-time charts are displayed on the ThingSpeak dashboard under the channel ”Air Quality and Noise Pollution.” Field 1 plots air quality (PPM) and shows values fluctuating between 28–65 PPM with a sharp spike to 64 PPM around 10:25. Field 2 plots sound/noise levels showing a gradual decline from 47 dB down to 30 dB, with a sudden spike to 69 dB near 10:25 — corresponding to the Telegram alert event.





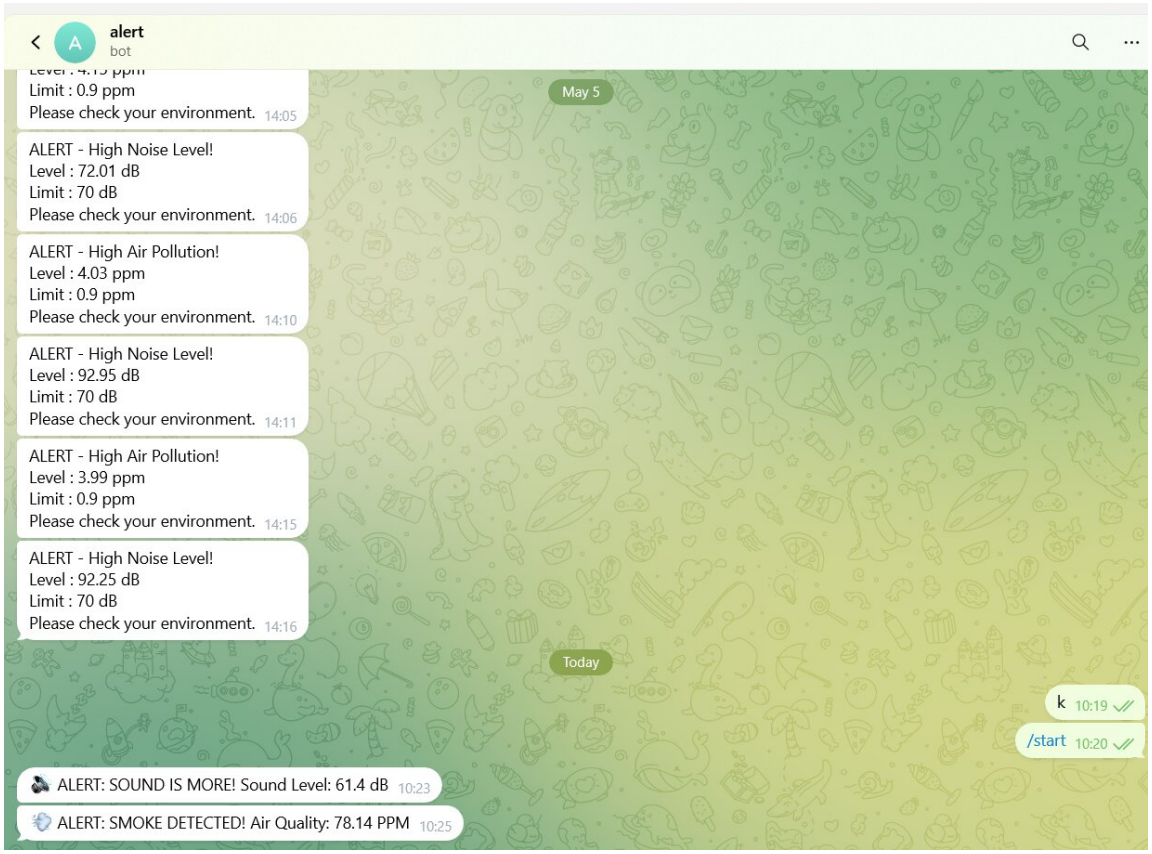
5.2 Serial Monitor Output

The serial monitor displays live readings every 5 seconds in the format: Sound Level: X dB — Air Quality: Y PPM followed by status labels (NORMAL or SMOKE DETECTED). One entry shows Air Quality: 78.14 PPM with status SMOKE DETECTED and Telegram sent — Status: 200, confirming the alert was dispatched. ThingSpeak responses (entry IDs like 23, 24) confirm successful cloud uploads. A 0 response indicates ThingSpeak’s rate limit was hit for that cycle

```
ThingSpeak Response: 23

Sound Level: 34.4 dB | Air Quality: 51.64 PPM
  Sound Status: NORMAL | Smoke Status: NORMAL
ThingSpeak Response: 0

Sound Level: 34.0 dB | Air Quality: 78.14 PPM
Telegram sent | Status: 200
  Sound Status: NORMAL | Smoke Status: SMOKE DETECTED
ThingSpeak Response: 0

Sound Level: 41.2 dB | Air Quality: 68.5 PPM
  Sound Status: NORMAL | Smoke Status: NORMAL
ThingSpeak Response: 24

Sound Level: 39.1 dB | Air Quality: 55.31 PPM
  Sound Status: NORMAL | Smoke Status: NORMAL
ThingSpeak Response: 0

Sound Level: 38.9 dB | Air Quality: 50.06 PPM
  Sound Status: NORMAL | Smoke Status: NORMAL
ThingSpeak Response: 0

Sound Level: 39.7 dB | Air Quality: 46.27 PPM
  Sound Status: NORMAL | Smoke Status: NORMAL
```

5.3 Telegram chat

The Telegram chat with the bot named ”alert” shows a history of notifications. On the current day, the bot received alerts: ”ALERT: SOUND IS MORE! Sound Level: 61.4 dB” at 10:23 and ”ALERT: SMOKE DETECTED! Air Quality: 78.14 PPM



5.4 summary of Analysis

The MATLAB analysis outputs a text summary captured at 07-May-2026 05:19:06. Both Sound Alert and Smoke Alert are marked NO , confirming safe conditions during that session. Sound averaged 31.69 dB (min: 26.93, max: 40.65) and smoke averaged 29.15 PPM (min: 28.44, max: 29.42)

```
=== SUMMARY ===
Analysis Time: 07-May-2026 05:19:06
Sound Alert: NO ✓
Smoke Alert: NO ✓
```

```
=== SOUND ANALYSIS ===
Average: 31.69 dB
Max: 40.65 dB
Min: 26.93 dB

=== SMOKE ANALYSIS ===
Average: 29.15 PPM
Max: 29.42 PPM
Min: 28.44 PPM
```

CONCLUSION

The Air and Noise Pollution Monitoring System was successfully designed and implemented using the Raspberry Pi Pico W as the central microcontroller. The system effectively integrates the MQ135 gas sensor and sound sensor module to continuously monitor air quality and ambient noise levels in real time. The collected data is transmitted to the ThingSpeak cloud platform via the MQTT protocol over WiFi, where it is stored, visualized as graphical plots, and monitored for threshold-based alerts. The use of MicroPython firmware and Thonny IDE made the development process efficient and straightforward. The system demonstrated stable and continuous operation during testing, with sensor readings being successfully published to the ThingSpeak cloud platform at regular intervals without any significant data loss or communication failures. The MQ135 gas sensor consistently detected variations in air quality corresponding to changes in the surrounding environment, while the sound sensor module accurately captured ambient noise level fluctuations in real time. The real-time graphical plots generated on the ThingSpeak dashboard provided a clear and intuitive visualization of both air quality and noise pollution trends over time, enabling easy interpretation of the environmental data. The dual-parameter nature of this system sets it apart from conventional single-parameter monitoring solutions, as it provides a correlated view of both air quality and noise pollution from a single compact and affordable node. This makes the system particularly well-suited for deployment in smart city infrastructure, residential areas, industrial zones, and educational IoT research environments. The low hardware cost, minimal power consumption, and compact form factor of the Raspberry Pi Pico W based design further enhance its suitability for large-scale distributed deployment across multiple monitoring locations. This low-cost, lightweight, and scalable IoT-based solution demonstrates the potential of embedded systems in addressing real-world environmental challenges. As a future enhancement, the system can be extended by integrating additional sensors such as the BME280 for temperature and humidity monitoring, the PMS5003 for precise PM2.5 and PM10 particulate matter measurements, and a GPS module for geo-tagged pollution mapping. Furthermore, machine learning algorithms can be applied to the historical data collected on ThingSpeak to predict pollution trends, identify pollution hotspots, and generate automated early warning alerts, making the system even more powerful and impactful for urban environmental management.

Bibliography

[1] Raspberry Pi Foundation, ”Raspberry Pi Pico W Datasheet,” Raspberry Pi Ltd., Cambridge, UK, Published: June 2022. Available: https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html

[2] MathWorks, ”ThingSpeak IoT Analytics Platform Documentation,” MathWorks Inc., Natick, MA, USA, Published: January 2023.

[3] OASIS Standard, ”MQTT Version 3.1.1 Protocol Specification,” OASIS Open, Published: October 2014.